Dependency Injection in ASP.NET Core Web API

Posted on July 13, 2020 by Dotnet Tutorials

Dependency Injection in ASP.NET Core Web API

In this article, we will discuss what is Dependency Injection and how to implement Dependency Injection in ASP.NET Core Web API with the help of a real-time example. Please read our previous article before proceeding to this article.

Dependency Injection in ASP.NET Core Web API

Dependency Injection is a technique that allows us to create dependent objects outside of a class and provide those objects to a class through constructors or properties. The purpose of using this is to make our code loosely coupled and easier to test.

Introduction to Inversion of Control and Dependency Injection

Inversion of Control: Inversion of Control is a design principle in which the control flow of a program is inverted. Instead of the programmer controlling the flow of a program, the flow is controlled by an external source. Inversion of Control is a key part of all assemblies that are intended to be reusable.

Dependency Injection: Dependency Injection is a design pattern that implements Inversion of Control for resolving dependencies. Dependency Injection is a set of software components and services that are used to create more loosely coupled applications.

ASP.NET Core has built-in Dependency Injection, which is a framework based, strong, out of the box Dependency Injection. The Dependency Injection framework includes three components:

- Service Container
- Service Provider
- Service Lifetime

Service Container

What is Dependency Injection IoC Container

Service Container is a collection of services and provides services to the client upon request. Service Container is responsible for:

- Knowing how to create the service
- Managing the lifetime of the service
- Providing the service to the client

Why Dependency Injection Inversion of Control Container

Dependency Injection IoC Container provides the following benefits:

- It provides a centralized place for managing the creation of dependent objects.
- It provides a centralized place for managing the lifetime of dependent objects.
- It provides a centralized place for managing the dependencies between objects.

Why Dependency Injection Inversion of Control Container

Dependency Injection IoC Container provides the following benefits:

- It provides a centralized place for managing the creation of dependent objects.
- It provides a centralized place for managing the lifetime of dependent objects.
- It provides a centralized place for managing the dependencies between objects.

Service Provider

Service Provider: The IoC container provides the services to the client through a service provider. Service Provider is responsible for providing the requested service to the client.

Service Lifetime

Service Lifetime Management Inversion of Control Container

Service Lifetime is the duration for which the service instance is valid for the client. The IoC container manages the lifetime of the service instance by creating and disposing it as per the configuration.